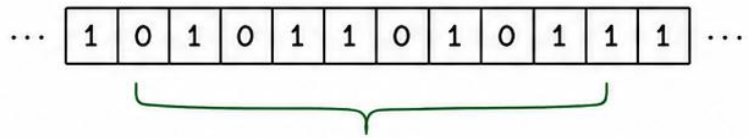
DGIM Algorithm

DGIM ALGORITHM (Datar-Gionis-Indyk-Motwani)

ESTIMATING THE NUMBER OF 1's IN THE LAST N BITS OF A BINARY STREAM

1. PROBLEM

- We are given an infinite stream of bits (0 or 1).
- At any time, we want to know how many 1's are present in the last N bits (sliding window).
- We must use very small memory (sublinear space).



2. KEY IDEA

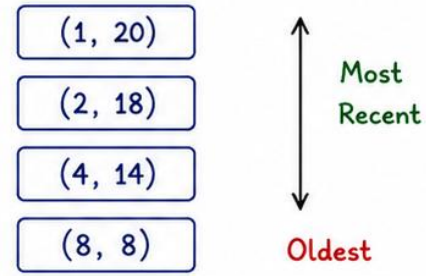
- Instead of storing every bit, DGIM stores a set of "buckets".
- Each bucket represents a block of consecutive 1's.
- A bucket is represented as (size, time) where time = position (timestamp) of the newest 1 in that bucket.
- Bucket sizes are powers of 2:

  1, 2, 4, 8, 16, 32, ...

- For each size, we keep at most TWO buckets.
- When a THIRD bucket of same size appears, merge the TWO OLDEST into one bucket of double the size.

3. BUCKET REPRESENTATION

Example:

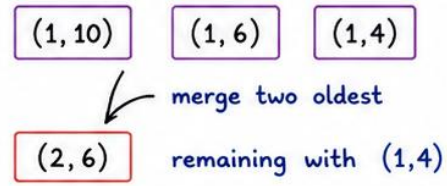


(8, 8) means:
There are 8 consecutive 1's and the rightmost (newest) 1 is at time 8.

4. DGIM RULES

1. Bucket sizes are powers of 2.
2. At most TWO buckets are allowed for each size.
3. When a THIRD bucket of the same size appears, merge the TWO OLDEST buckets of that size into one bucket of double the size.

Example (for size = 1):



5. FULL NUMERICAL EXAMPLE (STEP-BY-STEP) – DGIM ALGORITHM

Let Window Size N = 16

t :	1	2	3	4	5	6	7	8
bit :	1	0	1	1	0	1	1	1

(size, time) → (bucket size, timestamp of newest 1 in that bucket)

Most recent → right side

Oldest → left side

Step	t	Bit	Action	Buckets (Most recent → Oldest)	Explanation
1	1	1	Insert 1 → create bucket (1, 1)	(1, 1)	First 1 arrives.
2	2	0	Bit 0 → no change	(1, 1)	0 is ignored.
3	3	1	Insert 1 → create bucket (1, 3)	(1, 3) (1, 1)	Two buckets of size 1 → OK.
4	4	1	Insert 1 → create bucket (1, 4) Three size-1 buckets → merge two oldest (1, 1) & (1, 3) → new bucket (2, 3)	(1, 4) (1, 3) (1, 1) → (2, 3)	After merging two oldest size-1 buckets.
5	5	0	Bit 0 → no change	(1, 4) (2, 3)	0 is ignored.
6	6	1	Insert 1 → create bucket (1, 6)	(1, 6) (1, 4) (2, 3)	Two buckets of size 1 → OK.
7	7	1	Insert 1 → create bucket (1, 7) Three size-1 buckets → merge two oldest (1, 4) & (1, 6) → new bucket (2, 6)	(1, 7) (1, 6) (1, 4) (2, 3) → (2, 6)	Merge two oldest size-1 buckets.
8	8	1	Insert 1 → create bucket (1, 8)	(1, 8) (1, 7) (2, 6) (2, 3)	Two buckets of size 1 → OK.

Key Takeaways

- Bucket sizes are powers of 2 (1, 2, 4, 8, ...)
- At most TWO buckets of each size.
- When a THIRD bucket of same size appears, merge the TWO OLDEST into one bucket of double the size.
- We always keep timestamps of the newest 1 in each bucket.

□ = Size 1 bucket

□ = Size 2 bucket

□ = Merged result

□ = Older / larger bucket

Example:
(2,6) means → 2 consecutive 1's ending at time 6 (i.e., positions 5 and 6).

DGIM ALGORITHM – PART 2

6. CONTINUATION (t = 9 to 16)

Size 1 | Size 2 | Size 4 | Size 8

Most recent → Right Oldest → Left

Step	t	Bit	Action	Buckets (Most recent → Oldest)
9	9	0	Bit 0 → no change	(1,8) (1,7) (2,6) (2,3)
10	10	1	Insert 1 → create (1,10) Three size-1 buckets → merge two oldest (1,7) & (1,6) → new bucket (2,8)	(1,10) (1,8) (1,7) (2,6) (2,3) (1,10) (2,8) (2,6) (2,3)
11	11	1	Insert 1 → create (1,11)	(1,11) (1,10) (2,8) (2,6) (2,3)
12	12	0	Bit 0 → no change	(1,11) (1,10) (2,8) (2,6) (2,3)
13	13	1	Insert 1 → create (1,13) Three size-1 buckets → merge (1,10) & (1,11) → new (2,11)	(1,13) (1,11) (1,10) (2,8) (2,6) (2,3) (1,13) (2,11) (2,8) (2,6) (2,3)
14	14	1	Insert 1 → create (1,14)	(1,14) (1,13) (2,11) (2,8) (2,6) (2,3)
15	15	1	Insert 1 → create (1,15) Three size-1 buckets → merge (1,13) & (1,14) → new (2,14)	(1,15) (1,14) (1,13) (2,11) (2,8) (2,6) (2,3) (1,15) (2,14) (2,11) (2,8) (2,6) (2,3)
16	16	0	Bit 0 → no change	(1,15) (2,14) (2,11) (2,8) (2,6) (2,3)

Final Buckets

(Most recent → Oldest)

(1,15) (2,14) (2,11) (2,8) (2,6) (2,3)

Most recent ← Oldest

7. QUERY EXAMPLE

Query: How many 1's are in the last N = 10 bits
(i.e., from time 7 to time 16) ?

Current time t = 16

Window = [7 ... 16]

DGIM QUERY RULE

1. Take all buckets completely inside the window.
2. For the OLDEST bucket that is partially inside, take only HALF of it.
3. Sum all these.

Bucket (size,time)	Time Range Covered	Inside Window [7...16]?	Count to Add
(1, 15)	{15}	Yes (complete)	1
(2, 14)	{13, 14}	Yes (complete)	2
(2, 11)	{10, 11}	Yes (complete)	2
(2, 8)	{7, 8}	Yes (complete)	2
(2, 6)	{5, 6}	Partially (only 7 is inside)	Half of 2 = 1
(2, 3)	{1, 2}	No (outside)	0

- Yes (complete): Entire bucket lies inside [7...16].
- Partially: Bucket partly overlaps (oldest such bucket).
- No (outside): Bucket lies completely outside.

Estimated number of 1's in last 10 bits =

1 + 2 + 2 + 2 + 1 + 0 = 8

- 1 + 2 + 2: Buckets completely inside (added fully)
- 2 + 1: Oldest partially inside (take half)
- 0: Outside window (ignore)

8. COMPLEXITY



- Space Complexity : O(log N) buckets
- Update Time : O(1) amortized
- Query Time : O(log N)
- One-pass algorithm
- Deterministic
- Approximation error ≤ 50% of the size of the oldest bucket

9. ADVANTAGES



- Very small memory usage
- One-pass (streaming) algorithm
- Fast updates and queries
- Works well for high-speed streams
- Simple and efficient

10. LIMITATIONS



- Provides approximate answer
- Error up to 50% of the size of the oldest bucket
- Designed for counting 1's only
- Cannot reconstruct the stream

11. APPLICATIONS



- Network traffic monitoring
- Web click stream analysis
- Sensor and IoT monitoring
- Real-time dashboards
- Click fraud detection
- Intrusion detection systems



DGIM Algorithm provides efficient approximate counting of 1's in the last N bits using sublinear space and is ideal for high-speed data streams.